MANASA full project audit

Audit date: 2026-10-28
Scope: Current checkout at `4ff6846` (`atWork`). Read-only application, schema, security, UI, and test review. No application/configuration/test/database files were changed. The only added file is this report.

Executive summary

The current source provides a .NET 9 MVC learning-platform foundation: public student registration and login, administrator CRUD for students/teachers/subjects/courses/packages/quizzes, a published-course catalog, quiz attempts/results/history, and quiz-based XP, streaks, and badges. App and test project compile separately; the five current unit tests pass.

The main product gap is that “course access” currently means viewing published course descriptions and taking published quizzes. There are no lesson, enrollment, purchase, or course-progress entities/routes. Other confirmed gaps are unverified email registration, no Teacher area/workflow, and an incorrect CSS property that breaks the intended three-line course-description clamp. Database/runtime and browser behavior remain unverified.

Project and feature inventory

Area	Status	Evidence / scope
ASP.NET Core MVC, .NET 9, EF Core SQL Server, Identity	Working (source/build)	`Manasa.csproj`, `Program.cs`; app project builds.
Public registration, login, logout, student role assignment	Working (source); unverified (runtime)	`Controllers/AccountController.cs`; anti-forgery on POST, generic sign-in failure, local-only return redirects.
Email verification, password reset/change, MFA	Incomplete	Registration creates a usable account without confirmation; no verification/reset/change endpoints or mail service found.
Admin dashboard and management	Working (source); unverified (runtime)	Admin-only role gate on all area controllers; CRUD for students, teachers, subjects, courses, packages, quizzes.
Teacher account/profile	Incomplete	Admin can create teacher accounts and subject assignments; no Teacher area/controller/teaching

Area	Status	Evidence / scope
		workflow exists.
Course publishing and student catalog/details	Working (source); unverified (runtime)	Admin toggles publish state; student catalog/details filter published courses. This does not provide course content access.
Enrollment, package purchase/access entitlement	Incomplete	Package metadata exists, but no enrollment, order, payment, entitlement, or access-check implementation found. All students can see all published courses/quizzes.
Lessons and course progress	Incomplete	No lesson/content/progress model, routes, or services found. Quiz progress is separately tracked.
Quiz authoring, publishing, grading, attempts, results/history	Working (source; unit-tested grading)	Admin question/quiz management and student attempt flows exist. Server grades against saved snapshots and student rendering strips correct-answer flags.
Quiz gamification	Working (source); unverified (DB/runtime)	XP awards, badges, activity dates/streaks; idempotency indexes in `Data/ApplicationDbContext.cs`. One quiz pass awards XP once per quiz.
English/Arabic localization and RTL	Working (source); unverified (visual)	`Program.cs` supports `en` / `ar`; both shared resource files exist; layout sets `lang` / `dir` and selects Bootstrap RTL.
Automated tests	Incomplete	One test class, five unit cases for `QuizGrader`; no controller, Identity, EF/migration, authorization, localization, or UI tests.

Findings

Severity: **High** = material account/security or core data/workflow risk; **Medium** = significant product or operational gap; **Low** = limited UI/maintainability impact.

Medium — student registration accepts unverified email addresses

`Register` creates the account and immediately grants `Student`; it does not send a confirmation challenge. `RequireUniqueEmail` is configured, but confirmed-email sign-in is not required. A registrant can therefore create an account under another person's address and occupy it. Evidence: `Controllers/AccountController.cs` registration flow; `Program.cs` Identity configuration.

Recommendation: add a verified email flow before treating email as an owned identity, or explicitly use an administrator-provisioned account policy.

Medium — course/package workflow has no enrollment or entitlement enforcement

`Areas/Student/Controllers/DashboardController.cs` lists every published course, and `Areas/Student/Controllers/QuizzesController.cs` lists every quiz whose course and quiz are published. Package records store price and duration, but no purchase/enrollment tables or entitlement checks exist. This is confirmed incomplete behavior if packages are intended to control paid or assigned access.
Recommendation: define enrollment/package-access rules and enforce them in both course and quiz queries before adding lesson delivery.

Medium — lessons and learning progress are absent

No lesson/content or course-progress model, controller, or service exists; student course details return descriptive metadata only. Current "progress" is quiz-attempt activity/gamification. **Recommendation:** define the lesson/content and completion model, then connect student course routes and progress reporting.

Medium — teacher workflow ends at account/profile setup

Admin can create teacher accounts, assign subjects, and create teacher profiles, but there are no teacher-area endpoints. Teacher accounts can authenticate but have no teaching workflow in this repository.
Recommendation: either implement the intended teacher authoring/management workflow or avoid presenting these accounts as active teaching accounts until available.

Medium — course activity day and streak use UTC, not the user's local day

`Services/QuizService.cs` hardcodes `ActivityTimeZone = TimeZoneInfo.Utc` for activity-date writes and streak calculations. For users outside UTC, a quiz around local midnight can count toward an unexpected calendar day/streak. **Recommendation:** establish the product timezone policy (per-user or platform timezone) and use it consistently for both recording and summary calculations.

Low — course description truncation uses a misspelled CSS property

`wwwroot/css/site.css:164` uses `-webkit-box-oient: vertical`; the property is `-webkit-box-orient`. The intended three-line clamp can fail in engines relying on the WebKit box clamp behavior. **Recommendation:** correct the property and verify card layout in supported browsers.

Low — account setup token is carried in a URL

Admin-created student/teacher setup links put the bearer token in a query string (`Areas/Admin/Controllers/StudentsController.cs`, `TeachersController.cs`; accepted by `Controllers/AccountController.cs`). The custom provider limits validity to one hour (`Data/AccountSetupTokenProvider.cs`), reducing exposure, but URLs can still enter browser history,

proxy/server logs, or copied messages. **Recommendation:** use a one-time setup exchange that removes the secret from the resulting URL, and ensure operational logging/redaction policy covers query strings.

Security review

- **Confirmed controls:** area controllers use `[Authorize(Roles = "Administrator"/"Student")]`; all inspected modifying POST actions use anti-forgery validation; login redirects only to local return URLs; student quiz attempts/results are scoped to the current student; quiz grading is server-side. Authentication cookies are HTTP-only, SameSite Lax, eight-hour sliding expiration, and Secure in non-development (`Program.cs`).
- **Confirmed issue:** registration accepts unverified email (above). Identity lockout is enabled on created accounts and login requests lockout on failure, but there is no observed IP/device rate limiting.
- **No hard-coded passwords found in tracked app settings/source.** Development seed passwords are read from configuration keys and are not embedded. A second development-only administrator seed identity uses a nonstandard address/username; validate that this is intentional before supplying its secret in a shared development environment (`Data/DevelopmentDataSeeder.cs`).
- **Unverified:** production TLS/proxy configuration, Data Protection key persistence/shared-key setup, secret injection, request/query logging, deployed authorization behavior, and actual administrator credentials.

Database and data integrity

- SQL Server schema is migration-managed. Current migrations cover Identity, student username sequence, unique normalized email, catalog entities, quiz questions/options/attempt snapshots, XP awards, badges, and learning days.
- EF config includes unique constraints for normalized email, attempt numbering, one answer per attempt/question, one XP event key per student, unique student badges, and one activity row per student/day (`Data/ApplicationDbContext.cs`). Attempt snapshots preserve quiz text/options/settings when the author later edits content.
- Development startup runs `Database.MigrateAsync()` and seeds roles/demo accounts; production startup does not auto-apply migrations (`Program.cs`). This is a deployment responsibility, not evidence of a live schema state.
- **Unverified:** no database was connected or queried; migration application, SQL Server compatibility, existing production data, migration drift, indexes under real load, and backup/restore were not checked. No database writes were made.

UI, localization, RTL, and accessibility

- Shared layout has `lang` / `dir` , a skip link, navigation labels, and role-aware links. Forms inspected have labels/validation; result and achievement meters expose progressbar semantics. `prefers-reduced-motion` CSS exists.

- Arabic resource file and RTL Bootstrap/CSS paths exist. A code-only review cannot establish translated-string completeness, Arabic typography, mixed-direction rendering, responsive behavior, contrast, keyboard behavior, or screen-reader usability. No browser or assistive-technology checks were run.
- Confirmed CSS clamp typo is listed above. No broader visual defect is claimed without rendering the pages.

Build and test results

- `dotnet build Manasa.csproj --no-restore --verbosity:minimal` — **passed**, 0 warnings, 0 errors.
- `dotnet test Manasa.Tests/Manasa.Tests.csproj --no-restore --verbosity:normal` — **passed**, 5/5 tests. The test build emitted `NU1900` because NuGet vulnerability metadata could not be reached at `api.nuget.org`; package vulnerability status was not refreshed.
- `dotnet build Manasa.sln --no-restore --verbosity:minimal` — **failed** at `ResolveBuildCompressedStaticWebAssets` with 0 reported errors. The app project and test project build independently; root cause for this solution-level SDK/static-web-assets failure was not identified. Re-run in a clean build environment and investigate before release.
- No browser tests, live DB checks, migration application, security scans, accessibility scans, or load tests were run.

Prioritized remediation plan

1. **Security and account safety:** introduce email ownership verification (or a clear admin-provisioned alternative); inspect production secret, TLS, Data Protection, and URL-log handling; consider rate limiting anonymous registration/login.
2. **Data/workflow correctness:** decide and implement enrollment/package entitlement before course/quiz access; define lesson/content and progress workflows; decide intended Teacher capabilities; define timezone semantics for streaks.
3. **UI quality:** fix the CSS line-clamp property; visually verify English/Arabic RTL and LTR layouts, responsive pages, forms, quiz timer/result screens, keyboard navigation, and screen-reader labels.
4. **Automated coverage:** add integration tests for registration/confirmation, role authorization, admin CRUD/publishing, student isolation, quiz lifecycle/concurrency/limits, migration/model behavior, localization, and enrollment rules; add browser and accessibility coverage.
5. **Build/release validation:** diagnose the solution-level static-web-assets build failure in a clean environment; verify migrations against a disposable SQL Server database and document production migration/key-management steps.